23

Yoneda

Bringing together all that we've done in one of the pinnacles of abstraction in category theory.

We have now climbed to a certain height of abstraction, and it's time to pause to take in a breathtaking view of all of the country laid out before our eyes. In a way this chapter is an aside; you could move smoothly from the previous chapter to the following one about higher dimensions. For me, the abstractions in this chapter are one of the great joys of category theory. However, I will warn you that it is very formal and abstract, and that if it is the applications and examples that interest you this might just seem like too much formalism to you. But if you can get anything out of it at all, you have definitely started to appreciate the joy of abstraction.

23.1 The joy of Yoneda

We're going to look at one of the most important and beautiful uses of natural transformations. It is very abstract and formal but encapsulates some of the most deep fundamental truths about algebraic structures, while somehow at the same time being almost trivial to the point of being barely any more than type-checking. This apparent triviality is something that leads some people to dismiss category theory as contentless, but leads some other people to adore its ability to cut through to the core so cleanly that all mess seems to fall away, leaving just a fundamental nugget of simple but profound truth. As a friend of mine[†] put it: anyone can complicate things, but it takes real intelligence to simplify them. And I will note that simplifying something is not the same as what I'll call "simplisticating" them; that is, there is a difference between making something simple and making it simplistic.

The term "Yoneda" is named after Japanese mathematician Nobuo Yoneda. It may refer to a functor called the Yoneda embedding, and also a result called the Yoneda Lemma. It is so fundamental and ubiquitous that sometimes I joke

[†] Thank you Tyen-Nin Tay.

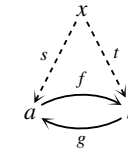
with category theory friends that the whole point of our research is to find the sense in which any particular thing is an instance of Yoneda.†

We have been hinting at Yoneda at various times in earlier chapters. We also mentioned that Yoneda would be an important use of contravariant functors, and we are now going to see that it's an important use of functor categories.

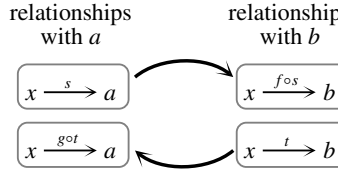
23.2 Revisiting sameness

In Section 14.4 we looked at the sense in which a category sees isomorphic objects as "the same". We're now going to revisit it at a higher level of abstraction which explains and encapsulates more of what is going on there.

In that section, we drew diagrams like the one on the right, with an isomorphism between a and b exhibited by inverses f and g. We considered an object x "looking" at a and b, and saw that x has the same relationships with a as it does with b.



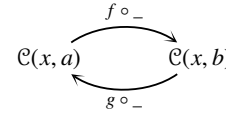
The correspondence between how x interacts with a and with b is shown informally on the right.



I have drawn this suggestively to look like yet another diagram exhibiting an isomorphism, with inverse arrows going in each direction. And in fact it's not just a schematic diagram: it's something we now have the abstract technology to make precise.

First assume that we're in a category $\mathcal{C}$ that is locally small, so that we really do have sets of morphisms. Now what I informally called the "relationships of x with a" is really the set $\mathcal{C}(x, a)$, and likewise the "relationships with b" is really the set $\mathcal{C}(x, b)$.

The above schematic diagram then becomes a diagram of sets and functions as shown on the right. The notation needs some explaining.



The little horizontal line ("underscore") is a deliberate gap left for us to place a variable. The function $f \circ \_$ takes an element of $\mathcal{C}(x, a)$ as an input, that is, a morphism $x \xrightarrow{s} a$. The function tells us to put s where the underscore is, so the

† In fact the Australian school of category theory seems to be so good at this that we sometimes go further and joke that "Yoneda" is the Australian category theory version of Mornington Crescent, but that's a rather esoteric joke for listeners of BBC Radio 4 of a certain generation.

output is $f \circ s$. The function going backwards takes as input a morphism $x \xrightarrow{t} b$ and gives the output $g \circ t$. Thus these functions with underscores correspond to the large arrows shown informally in the previous diagram of "relationships". Note that we've gone up a level of abstraction here, and are now considering morphisms themselves as elements of sets, with functions acting on them.

This has the potential to be quite confusing because of the different levels involved. I remember being very mind-blown about it when I first encountered it. The way we annotate composition like $f \circ s$ "backwards" adds further potential confusion, but if in doubt I draw out all the morphisms including source and target and the type-checking sorts everything out.

Now, so far this just *looks* like a diagram showing an isomorphism of sets. We can check that the functions in question are really inverses by following their action on elements around. Here's the composite from $\mathcal{C}(x, a)$ to itself:

$$\begin{array}{ccccc} \mathcal{C}(x,a) & \xrightarrow{f\circ\_} & \mathcal{C}(x,b) & \xrightarrow{g\circ\_} & \mathcal{C}(x,a) \\ s & \longmapsto & f \circ s & \longmapsto & g \circ (f \circ s) \;=\; s \end{array}$$

For the last step, note that $g \circ (f \circ s) = (g \circ f) \circ s$ by associativity, and $g \circ f$ is the identity because f and g are inverses (the whole point of this story). So the composite from $\mathcal{C}(x, a)$ to itself sends s to s, so is the identity function.†

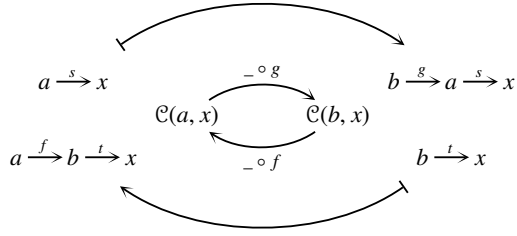
An analogous argument shows that the composite the other way round is the identity function $\mathcal{C}(x, b) \longrightarrow \mathcal{C}(x, b)$, so we have an isomorphism of sets as required. Note how this was a direct result of the morphisms f and g being inverses; in fact each half of the inverse definition for f and g produced half of the inverse definition at the level of function on the homsets.

Dually, we can think about morphisms *to* x rather than *from* x.

Things To Think About

T 23.1 Try the dual version: see if you can draw the diagram, write down the functions with underscores correctly, and check the action on elements.

Here is the diagram for the dual version.



† You might prefer this to say "so it is the identity function". I'm using language in the spare way mathematics is often written, in case you want to go on to read more formal texts.

Note that the directions have switched:

When acting on morphisms *out* of x, we use *post-composition* with f. The direction of the resulting function is shown on the right.

$$\begin{array}{ccc} \mathcal{C}(x,a) & \xrightarrow{f\circ\_} & \mathcal{C}(x,b) \\ x \xrightarrow{s} a & \longmapsto & x \xrightarrow{s} a \xrightarrow{f} b \end{array}$$

When acting on morphisms *into* x, we use *pre-composition* with f. The direction of the resulting function is reversed.

$$\begin{array}{ccc} \mathcal{C}(b,x) & \xrightarrow{\_\circ f} & \mathcal{C}(a,x) \\ b \xrightarrow{s} x & \longmapsto & a \xrightarrow{f} b \xrightarrow{s} x \end{array}$$

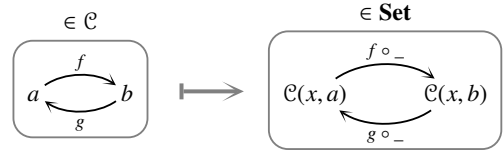
We are going to see that these situations are each part of a functor, and the switching of directions means the second (dual) one is *contravariant*.†

Expressing all this as functors will also make precise the sense in which an isomorphism between objects of $\mathcal{C}$ "induces" an isomorphism of homsets. The functors in question are called representable functors.

23.3 Representable functors

So far we have established that, given an isomorphism $a \xrightarrow{f} b$ in $\mathcal{C}$, we can produce an isomorphism on sets of morphisms as shown below.

An isomorphism in $\mathcal{C}$ produces an isomorphism in **Set**.



I hope that this suggestive diagram has caused you to wonder whether this is actually down to some sort of functor $\mathcal{C} \longrightarrow \mathbf{Set}$. This is the kind of situation in category theory where I think "everything you want to be true is true". (Of course, this depends on wanting the right sort of things.)

I am claiming there is a functor with an action as follows; note that here x is fixed, but now a and b can be any objects, and f any morphism (not necessarily an isomorphism). We will call this functor H^x to remind us that x is fixed.

	$\mathcal{C}$	$\xrightarrow{H^x}$	**Set**
on objects:	a	$\longmapsto$	$\mathcal{C}(x,a)$
on morphisms:	$a \xrightarrow{f} b$	$\longmapsto$	$\mathcal{C}(x,a) \xrightarrow{f\circ\_} \mathcal{C}(x,b)$

† The type of functor that switches the direction of morphisms.

Things get a little hairy here as there are so many levels at play, but if we proceed slowly we can show that this does indeed define a functor.

Things To Think About

T 23.2 So far all we have established is that for each morphism $f \in \mathcal{C}$, we have a function $f \circ \_$. See if you can work out what we need to check to show that this construction makes H^x a functor. (And then check it: however, I personally think working out what needs to be checked is the hard part.)

To check functoriality we need to keep our wits about us a bit but it's basically just type-checking. We just need to keep in mind that when we're looking at a function on homsets we'll probably want to examine it on elements, but the elements in that case are elements of homsets, that is, morphisms of $\mathcal{C}$.

First we need to check that H^x sends identities to identities.

The action of H^x on identities is shown on the right, so we need to check the function $1_a \circ \_$ is the identity function.

$$\begin{array}{ccc} \mathcal{C} & \xrightarrow{H^x} & \mathbf{Set} \\ a \xrightarrow{1_a} a & \longmapsto & \mathcal{C}(x,a) \xrightarrow{1_a \circ \_} \mathcal{C}(x,a) \end{array}$$

We check the action of $1_a \circ \_$ on an element $f \in \mathcal{C}(x,a)$ as shown on the right: f is sent to f so $1_a \circ \_$ is indeed the identity function.

$$\begin{array}{ccc} \mathcal{C}(x,a) & \xrightarrow{1_a \circ \_} & \mathcal{C}(x,a) \\ f & \longmapsto & 1_a \circ f = f \end{array}$$

For composition we consider a pair of composable morphisms $a \xrightarrow{f} b \xrightarrow{g} c$ in $\mathcal{C}$ and compare two things:

If we apply the functor H^x first and then compose the results in **Set** we get:

$$\begin{array}{ccccc} \mathcal{C}(x,a) & \xrightarrow{f\circ\_} & \mathcal{C}(x,b) & \xrightarrow{g\circ\_} & \mathcal{C}(x,c) \\ s & \longmapsto & f \circ s & \longmapsto & g \circ (f \circ s) \end{array}$$

If instead we compose in $\mathcal{C}$ first and then apply the functor H^x we get:

$$\begin{array}{ccc} \mathcal{C}(x,a) & \xrightarrow{(g\circ f)\circ\_} & \mathcal{C}(x,c) \\ s & \longmapsto & (g \circ f) \circ s \end{array}$$

The action of these two functions on s is the same, by associativity. So in fact, functoriality in this case comes down to associativity of composition in $\mathcal{C}$.

It is good practice to observe this relationship, rather than just ignoring the parentheses because we know that associativity holds. Ideologically this is because category theory is a discipline of seeing how and why structure arises; in practice it's important because if we move into situations in which associativity doesn't hold strictly, then it's important to know what the consequences are.

So far we have made a functor by fixing an object x and looking at sets $\mathcal{C}(x, a)$. The dual version looks at sets $\mathcal{C}(a, x)$ instead but, as we mentioned at the end of the last section, the functor is now contravariant, that is, it reverses the direction of morphisms.

Things To Think About

T 23.3 Try and follow through aaall of the above construction, making sure to take into account the functor now being contravariant.

As before we fix an object $x \in \mathcal{C}$, but this time we are making a functor that sends an object x to the set $\mathcal{C}(a, x)$. We are making a contravariant functor, that is, a functor $\mathcal{C}^{\text{op}} \longrightarrow \mathbf{Set}$. We will call this one H_x with the subscript (rather than the previous superscript H^x) indicating that we are now looking at homsets of morphisms *into* x. The functor H_x acts as follows (but note that as always I will only draw morphisms in $\mathcal{C}$, not in $\mathcal{C}^{\text{op}}$):

	$\mathcal{C}^{\text{op}}$	$\xrightarrow{H_x}$	**Set**
on objects:	a	$\longmapsto$	$\mathcal{C}(a, x)$
on morphisms:	$a \xrightarrow{f} b \in \mathcal{C}$	$\longmapsto$	$\mathcal{C}(b, x) \xrightarrow{-\circ f} \mathcal{C}(a, x)$

In fact it's the same construction as before, but starting with $\mathcal{C}^{\text{op}}$ instead of $\mathcal{C}$. We've just unraveled it using the definition $\mathcal{C}^{\text{op}}(x, a) = \mathcal{C}(a, x)$. Thus functoriality follows by waving the BOGOF[†] wand and saying "dually"; we don't have to check anything. It is perhaps just worth noting that identities are preserved because they act as identities on the other side this time.

These functors are sometimes written as $\mathcal{C}(x, \_)$ and $\mathcal{C}(\_, x)$ with the underscore or "blank" indicating to us where the variable goes. Otherwise we use H or h, perhaps standing for "hom".[‡]

In summary we have the following functors for a fixed object x.

$$\begin{aligned} \mathcal{C}(x, \_) = H^x \text{ or } h^x &: \mathcal{C} \longrightarrow \mathbf{Set} \\ \mathcal{C}(\_, x) = H_x \text{ or } h_x &: \mathcal{C}^{\text{op}} \longrightarrow \mathbf{Set} \end{aligned}$$

Terminology

These things are so widespread and formally useful that we give them names. We mentioned before that functors $\mathcal{C}^{\text{op}} \longrightarrow \mathbf{Set}$ are called *presheaves* on $\mathcal{C}$. When we use the prefix "pre-" in abstract math it's usually to indicate that the structure is a starting point for something more complicated that will need more conditions. In this case a presheaf is indeed the starting data for a sheaf.

[†] Buy One Get One Free, that is, prove one and get the dual for free.
[‡] Some people write the homset $\mathcal{C}(a, b)$ as $\hom(a, b)$ or $\hom_{\mathcal{C}}(a, b)$.

Functors of the form H^x and H_x are called *representable*; more accurately any functor isomorphic to one of these (via a natural isomorphism) is called representable. We'll come back to this. Representable functors are particularly well-behaved, so if a functor can be expressed in this way it's very handy.

Here is one thing we can immediately do using our newly coined functors H_x and H^x. We began all this by studying the fact that if f is an isomorphism then it induces an isomorphism on the homsets. That is, if f is an isomorphism then $H^x(f)$ is also an isomorphism, and dually so is $H_x(f)$. This is saying that the functors H^x and H_x *preserve* isomorphisms. But in fact all functors do that, as we saw in Chapter 20.

You might be feeling your head spinning at the different levels operating here, and I think that's quite a reasonable response. And you will either be excited or horrified that there is yet another level. You might have wondered why we kept having to fix an object x to start. That's a good instinct: this is itself part of a functor that takes an object x and sends it to the functor H^x (or dually to H_x). This is a functor sending objects of $\mathcal{C}$ to functors themselves, which means we need to invoke the categories of functors that we just met in the previous chapter. The functor sending x to H_x is the *[drumroll]* Yoneda embedding.

23.4 The Yoneda embedding

We will now make use of the functor categories we saw in the previous chapter: we saw that given categories $\mathcal{C}$ and $\mathcal{D}$ we have a category $[\mathcal{C}, \mathcal{D}]$ whose objects are functors $\mathcal{C} \longrightarrow \mathcal{D}$ and morphisms are natural transformations.

We now want to send an object $x \in \mathcal{C}$ to a functor $H_x \colon \mathcal{C}^{\mathrm{op}} \longrightarrow \mathbf{Set}$. But H_x is itself an object of the functor category $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$, so we can build a functor as follows. We are going to call it $H_\bullet$ where the "blob" $\bullet$ is like an empty spot for us to place the variable x, or f.

	$\mathcal{C}$	$\xrightarrow{H_\bullet}$	$[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$
on objects:	x	$\longmapsto$	H_x
on morphisms:	$x \xrightarrow{f} y$	$\longmapsto$	$H_x \xRightarrow{H_f} H_y$

To define this functor we need to define H_f. At this point it is important to remain calm[†] and trust that a little type-checking will see us through this.

Things To Think About

T 23.4 See if you can work out what the natural transformation H_f must do. Just note that like all natural transformations it must have one component for each object $a \in \mathcal{C}$. Remain calm about the fact that if you're defining H_f then x and y have been fixed. Write down what the endpoints of the component must be (by type-checking) and then see what is the only way you can construct a component in that place using the given information.

To define H_f, let's first see where its components must "live". Remember through all of this $x \xrightarrow{f} y$ is fixed.

We are defining a natural transformation $H_f \colon H_x \Longrightarrow H_y$, so by definition we must have components as shown on the right.

$$\begin{array}{lll} \forall\, a \in \mathcal{C}, & H_x(a) & \xrightarrow{(H_f)_a} H_y(a) \\ \text{that is,} & \mathcal{C}(a,x) & \longrightarrow \mathcal{C}(a,y) \\ & a \xrightarrow{s} x & \longmapsto \quad a \xrightarrow{?} y \end{array}$$

It remains to decide where an element $a \xrightarrow{s} x$ should be mapped to. Now it's like a type-checking jigsaw puzzle: we're starting with a morphism $a \xrightarrow{s} x$ and a morphism $x \xrightarrow{f} y$, and we're trying to produce a morphism $a \longrightarrow y$, so there's really only one thing we can do: compose s and f.

This gives us the following definition of the component of H_f at a.

$$\begin{array}{lll} \mathcal{C}(a,x) & \xrightarrow{(H_f)_a} & \mathcal{C}(a,y) \\ a \xrightarrow{s} x & \longmapsto & a \xrightarrow{s} x \xrightarrow{f} y \end{array}$$

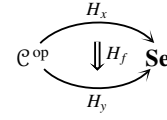
This might remind you of what we did when we were defining the action of the functor H_x on morphisms in the first place. This can get confusing because so many things get defined as post- or pre-composition, but if you remain calm and keep type-checking everything will stay in place.

Things To Think About

T 23.5 Can you check that this definition of H_f satisfies naturality?

Naturality is then largely another case of remaining calm and keeping the notation in place.

We are trying to define a natural transformation as shown on the right.



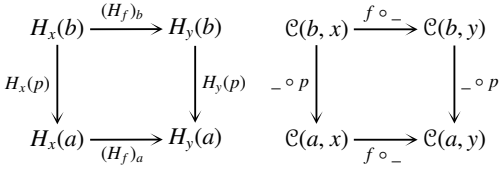
[†] My wonderful PhD supervisor Martin Hyland often reminds us to remain calm, and it's very sound advice in abstract math and also in life.

We need to check naturality squares, but we also need to remember that H_x and H_y are *contravariant* functors so they flip the direction of arrows. If you forget this then the square won't type-check, so you'll get a reminder.

Technically we have a naturality square for each morphism of $\mathcal{C}^{\mathrm{op}}$, but as usual I prefer never drawing morphisms in $\mathcal{C}^{\mathrm{op}}$, but keeping them drawn in $\mathcal{C}$ and flipping directions when relevant.

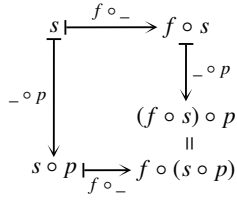
Recall that throughout this whole scenario we have fixed $x \xrightarrow{f} y$ in $\mathcal{C}$. Now we need a naturality square for every $a \xrightarrow{p} b$ in $\mathcal{C}$.

Writing out the definition of naturality square gives the first square here. (Remember that H_x and H_y flip the direction of p.) It evaluates to the second square shown.

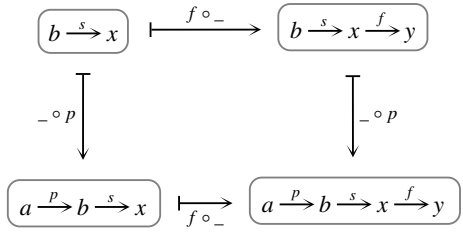


Once you've evaluated the corners of the square there's really no choice of what the morphisms can be, just by type-checking.

One way to check this commutes is to sort of blindly insert a variable at the top left corner and follow it around according to the formulae, as shown here. We see that going round in either direction gives the same answer by associativity of composition in $\mathcal{C}$.



However, if you want to type-check it properly (to feel better that we did everything right) you can draw all the morphisms out as shown here.



The first method is fine, but I think the second is safer and more illuminating: it helps us see that there was no choice as to which morphisms were pre-composed and which were post-composed. Drawing out the sources and targets forces us to compose things on the correct side, whereas if you just write strings like $f \circ s$ you could quite merrily write $s \circ f$ and never know that anything was wrong.

We have now constructed the key functor for Yoneda.

Definition 23.1 The functor $\mathcal{C} \xrightarrow{H_\bullet} [\mathbb{C}^{\mathrm{op}}, \mathbf{Set}]$ is called the *Yoneda embedding*.

We have slightly skipped ahead of ourselves here because we've called it an "embedding" before proving that it is an embedding; in fact we haven't even said what an embedding is.

Aside on embeddings

The idea of an embedding is that it's a bit like a subset inclusion, but one dimension up. For example, if we observe that the natural numbers are a subset of the integers, there's a function which just performs that inclusion sending every natural number to itself regarded as an integer. We often denote it like this: $\mathbb{N} \hookrightarrow \mathbb{Z}$, with a little "hook" at the beginning to remind us that it's like a subset inclusion $\subset$.

Philosophers may worry about whether the natural number n is actually the same as the integer n or not, but we can get around this by observing that the key is this function is injective, and then it doesn't really matter whether the elements of $\mathbb{N}$ are actually the same ones as in $\mathbb{Z}$ or not.

If we now go up to the level of categories we might want to encapsulate the idea of a *subcategory* inclusion, but now things get a bit hazy. Do we want strict subcategory inclusions or weak ones? That is, do we want to be strictly or weakly injective on objects? Do we want only full subcategory inclusions?

Regardless of quite what definition you take of an embedding, the "Yoneda embedding" is generally considered to be worthy of the name because it is full and faithful, as we'll now show.

Theorem 23.2 *The Yoneda embedding is full and faithful.*

This proof essentially falls into place by type-checking, if you remain calm. I find it enormously satisfying to re-prove it rather than look up a proof. It's like doing an abstract jigsaw puzzle again even though you've done it before: it's still satisfying to me no matter how many times I do it. (On the other hand if it's not satisfying to you the first time it might never be satisfying.) It does require one abstract "trick", which I prefer to think of as a general principle.

General Yoneda-y principle

When you're dealing with a functor $H_x = \mathcal{C}(\_, x)$ one thing you can always do in the absence of any special information is evaluate it at the object x, giving $H_x(x) = \mathcal{C}(x, x)$. And then after that, the one thing that you know exists in that homset is the identity 1_x. This is the principle behind all Yoneda-y[†] things: that

[†] "Yoneda-y", like "chocolatey".

the one thing we know we have in any world of homsets is the identity, and all the Yoneda functors and natural transformations are acting by composition on one side or the other. This encapsulates all the structure of a category, and thus everything else that follows is deeply inherent to the structure of a category. Typically we don't hold all the proofs in our brain, just this principle, and then everything else flows from it.

Things To Think About

T 23.6 See if you can prove Theorem 23.2 for yourself using my Yoneda-y principle. The whole thing is basically just unraveling definitions and type-checking. You might well get stuck, in which case I suggest reading through my proof but stopping once I've shown how to use that principle and then trying to proceed yourself. This is in general a good way to read math proofs: try and do it yourself, get stuck, read someone else's proof just to the point where they reveal a step you didn't get, then try and continue by yourself, get stuck again, and iterate.

Proof First let us show that $H_\bullet$ is faithful, which means "locally injective".

So we fix $x, y \in \mathcal{C}$, consider morphisms $f, g\colon x \longrightarrow y$ and check the implications shown on the right.

$$\begin{array}{rrcl} & H_\bullet(f) = H_\bullet(g) & \Longrightarrow & f = g \\ \text{that is} & H_f = H_g & \Longrightarrow & f = g \end{array}$$

Now, H_f and H_g are natural transformations, so being equal means all their components are equal, as functions.

Their respective components for each $a \in \mathcal{C}$ are shown on the right, and we are supposing that these are equal *for all* $a \in \mathcal{C}$.

$$\begin{array}{l} (H_f)_a\colon \mathcal{C}(a,x) \xrightarrow{f\circ\_} \mathcal{C}(a,y) \\ (H_g)_a\colon \mathcal{C}(a,x) \xrightarrow{g\circ\_} \mathcal{C}(a,y) \end{array}$$

Now we invoke the Yoneda-y principle: we can put $a = x$ and consider $1_x \in \mathcal{C}(x,x)$. We know $(H_f)_x$ has to be the same function as $(H_g)_x$, so they need to send everything to the same place, and in particular have to send 1_x to the same place. But one of them sends 1_x to f and the other to g.

Set $a = x$. The component $(H_f)_x$ is a function acting on 1_x as shown here.

$$\begin{array}{rcl} (H_f)_x : \mathcal{C}(x,x) & \xrightarrow{f\circ\_} & \mathcal{C}(x,y) \\ 1_x & \longmapsto & f \circ 1_x = f \end{array}$$

Similarly the component $(H_g)_x$ is a function acting on 1_x as shown here.

$$\begin{array}{rcl} (H_g)_x : \mathcal{C}(x,x) & \xrightarrow{g\circ\_} & \mathcal{C}(x,y) \\ 1_x & \longmapsto & g \circ 1_x = g \end{array}$$

By hypothesis[†] the components $(H_f)_x$ and $(H_g)_x$ are equal as functions; this means they have to produce the same result on every element of $\mathcal{C}(x, x)$, and in particular on 1_x. So $f = g$ as required.

This proof went in steps in a process of gradually homing in on the one crucial piece of information at 1_x, like this:

1. $H_f = H_g$ by hypothesis.
2. So they're equal at every component $(H_f)_a = (H_g)_a$; in particular $(H_f)_x = (H_g)_x$.
3. These are equal as functions, so in particular $(H_f)_x(1_x) = (H_g)_x(1_x)$.
4. But if we evaluate these we find that the left is f and the right is g so we can conclude $f = g$ as required.

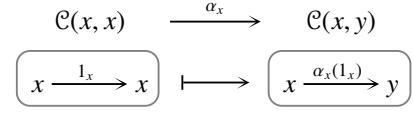
If you read a proof in a standard textbook you are quite likely to find that the proof of faithfulness consists of one line, essentially the line (3) above. Once you are able to type-check and unravel proficiently, that one line is enough!

We will now show that $H_\bullet$ is full. By definition this means: any natural transformation $\alpha\colon H_x \Longrightarrow H_y$ must be of the form H_f for some $x \xrightarrow{f} y$. So first we need to produce a morphism f from the natural transformation α.

There are two ways to proceed here. One way is to follow your nose and find the only morphism $x \longrightarrow y$ that falls out of the information we have so far. Another way is to think about what we've already done: we discovered that $(H_f)_x(1_x) = f$, which is saying $\alpha_x(1_x) = f$ for the case $\alpha = H_f$; perhaps that works more generally? Both of these approaches produce the same result.

Now α has components for all $a \in \mathcal{C}$, $\boxed{\alpha_a\colon \mathcal{C}(a, x) \longrightarrow \mathcal{C}(a, y).}$ These α_a are functions, and we can use the same "Yoneda-y" principle:

The one thing we know we have here is the identity $1_x \in \mathcal{C}(x, x)$. So we can look at the action of α_x on 1_x as shown on the right.



We have produced a "canonical" morphism $x \longrightarrow y$ from α, so we can just try it: set $f = \alpha_x(1_x)$ and claim that $\alpha = H_f$.

At this point if you did the "follow your nose" method, you might think there's no earthly reason to suspect that α should equal H_f; this is where going back to $(H_f)_x(1_x) = f$ helps. That is, it worked in the case $\alpha = H_f$ so we are in with a fighting chance here.

[†] This means "using what we assumed at the start of this proof", which in this case is $H_f = H_g$.

To show that $\alpha = H_f$ we need to show that all their components are equal, that is, for all $a \in \mathcal{C}$, $\alpha_a = (H_f)_a$.

But $(H_f)_a$ acts as shown here.

$$\begin{array}{ccc} \mathcal{C}(a,x) & \xrightarrow{(H_f)_a \,=\, f\circ\_} & \mathcal{C}(a,y) \\ p & \longmapsto & f \circ p \end{array}$$

So we need to show that for any morphism $a \xrightarrow{p} x$, we have $\alpha_a(p) = f \circ p$.

The information we haven't yet used about α is the naturality, so it's just as well to write down what a naturality square looks like: for any morphism $a \xrightarrow{p} b$ the square on the right commutes.

$$\begin{array}{ccc} \mathcal{C}(b,x) & \xrightarrow{\alpha_b} & \mathcal{C}(b,y) \\ {\scriptstyle \_\circ p}\downarrow & & \downarrow{\scriptstyle \_\circ p} \\ \mathcal{C}(a,x) & \xrightarrow[\alpha_a]{} & \mathcal{C}(a,y) \end{array}$$

Now we use the Yoneda-y principle again: the one thing we know exists is the element $1_x \in \mathcal{C}(x,x)$. We also know we are trying to prove something for all morphisms $a \xrightarrow{p} x$. This is all pointing towards setting $b = x$ in this diagram and seeing what happens.

Set $b = x$ in the above naturality square. Then for any morphism $a \xrightarrow{p} x$ we have the naturality square shown here.

$$\begin{array}{ccc} \mathcal{C}(x,x) & \xrightarrow{\alpha_x} & \mathcal{C}(x,y) \\ {\scriptstyle \_\circ p}\downarrow & & \downarrow{\scriptstyle \_\circ p} \\ \mathcal{C}(a,x) & \xrightarrow[\alpha_a]{} & \mathcal{C}(a,y) \end{array}$$

These are sets and functions, so we can see what happens to a particular element. If we start from the identity $1_x \in \mathcal{C}(x,x)$ and follow it around the diagram in two ways we get the result shown here.

$$\begin{array}{ccc} 1_x & \xrightarrow{\alpha_x} & \alpha_x(1_x) = f \\ {\scriptstyle \_\circ p}\downarrow & & \downarrow{\scriptstyle \_\circ p} \\ & & f \circ p \\ & & \| \\ 1_x \circ p \;=\; p & \xrightarrow[\alpha_a]{} & \alpha_a(p) \end{array}$$

Naturality tells us both ways round the square are equal, so we can deduce that $\alpha_a(p) = f \circ p$ and thus that $\alpha = H_f$. So $H_\bullet$ is full as claimed. □

Aside on research and proofs

One of the reasons proving something new is harder than re-proving something you already know is true is that when it's new you don't actually know it's true until you've proved it. The best you can do is have a very strong hunch, either from seeing a lot of examples or (my preferred way) by feeling something structural going on in the situation. Whereas if you know that something is true we can do what we did above and say "Well the only way to construct this

f is like this, so it must be that". When you're doing research (or at least, when I'm doing research) I get a very strong hunch that something is true but if I run into trouble proving it then I can just end up in a big mess of doubt, and flip over to trying to prove it's not true. One can spend a lot of time flipping backwards and forwards like that. But the doubt is important because if you trust your hunch too much then you might just miss some subtle point in your conviction that the thing is true. Plenty of erroneous "proofs" have arisen in research like that. Sometimes it just means the proof has a hole but the result turns out to be still correct, but sometimes the result is actually wrong.

The proof that the Yoneda embedding is full and faithful is something I find supremely, joyfully satisfying in its own abstract right, but it is also of deep importance. The idea is that we can start with any category $\mathcal{C}$ and embed it in its presheaf category[†] via the Yoneda embedding $\mathcal{C} \xrightarrow{H_\bullet} [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$.

Now there is a general principle we mentioned before which is that the data for a natural transformation lives in the *target* category, and so the structure of a functor category is largely inherited from its target category. The target category for presheaves is the category **Set**, which is a particularly well-behaved category. This means that the category $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$ of presheaves inherits excellent structure from **Set**, even if $\mathcal{C}$ does not have that structure, for example limits and colimits.

The fact that $H_\bullet$ is an embedding means we can then sort of regard $\mathcal{C}$ as a subcategory of the presheaf category. Or at least, a copy of it appears as a subcategory of the presheaf category: we know that the embedding takes an object $x \in \mathcal{C}$ to the functor H_x, so if we take all the functors of this form and all the natural transformations between them, we get a copy (or model) of $\mathcal{C}$ as presheaves. Note that taking *all* the natural transformations between them is right because the embedding is full; if it weren't full then taking all the natural transformations would give us extra morphisms that are not in $\mathcal{C}$.

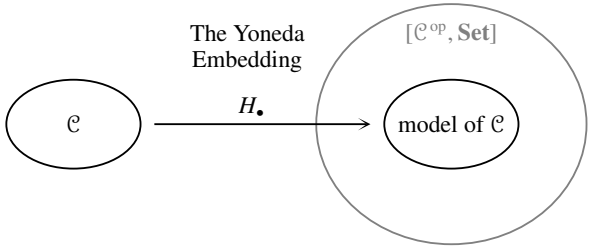
This is one of the reasons that presheaves of the form H_x are so important, and why they get a name: they are called *representable functors*. However, as these are objects in a category, it's better to define them only up to isomorphism, so functors are called representable more generally if they are isomorphic to one of these.

One of the brilliant things about the Yoneda embedding is that $\mathcal{C}$ is not just any old subcategory of $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$. Something highly canonical has happened in that $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$ is the *closure of* $\mathcal{C}$ *under colimits*. That is, we throw in all colimits for things in $\mathcal{C}$ and nothing more. So the parts of the presheaf category

---

[†] Recall: functors $\mathcal{C}^{\mathrm{op}} \longrightarrow \mathbf{Set}$ are called presheaves on $\mathcal{C}$.

that are not in the (model of) the subcategory $\mathcal{C}$ are all colimits of the things in $\mathcal{C}$ in a canonical way. The other side of this coin is then to say that *every presheaf is canonically a colimit of representables*. These are all very profound results in category theory, illustrated by the schematic diagram below.



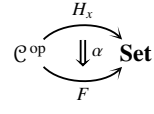
In fact, the principles we used in proving that the Yoneda embedding is full and faithful are part of a more general theorem which we will now describe.

23.5 The Yoneda Lemma

The Yoneda Lemma is called a "lemma" although it's really very profound, important, and ubiquitous. Usually something is called a lemma if it's just something slightly trivial that helps us prove something else. I like the fact that this makes the Yoneda Lemma sound self-deprecating. In a way it really is very trivial and the proof, though complex, is essentially just many layers of type-checking. However the ramifications are very widespread.

It essentially captures the "Yoneda-y principle" we used above, which was that our natural transformations kept being entirely determined by looking at the identity. We were looking at natural transformations $H_x \Longrightarrow H_y$ but the principle only really depended on the *source* functor being of the form H_x. The target functor could have been any functor $F\colon \mathcal{C}^{\,\mathrm{op}} \longrightarrow \mathbf{Set}$.

So we consider a natural transformation as shown on the right. By definition, it must have a component α_a for every object $a \in \mathcal{C}$, living here: $H_x(a) = \mathcal{C}(a, x) \xrightarrow{\alpha_a} F(a)$.



We can now use the Yoneda-y principle. We set $a = x$ and look at the action of α_x on $1_x \in \mathcal{C}(x, x)$, as shown on the right.

$$\begin{array}{ccc} \mathcal{C}(x,x) & \xrightarrow{\alpha_x} & F(x) \\ x \xrightarrow{1_x} x & \longmapsto & \alpha_x(1_x) \end{array}$$

The image[†] of 1_x, that is $\alpha_x(1_x)$, is now not a morphism, because $F(x)$ is a set, not necessarily a homset. But the idea is that in order to define a natural

[†] This means: where 1_x lands when we apply the function in question.

transformation α we have a free choice of where 1_x can go, and moreover this completely determines the rest of the components of α, by following a naturality square around as we did in the proof of $H_\bullet$ being full. Thus:

> natural transformations $H_x \Longrightarrow F$
> correspond precisely to
> elements of $F(x)$

This is essentially the content of the Yoneda Lemma, except that because we now understand the principles of category theory we know that "correspond precisely to" is a rather vague statement and sounds ambiguously a bit like it's just a bijection between sets. The things in this case don't just correspond, they correspond in a way that uses the identity and composition structure of the categories all over the place. That is in turn encapsulated by some naturality. Here is what the formal statement often looks like:

Theorem 23.3 (Yoneda Lemma) *Let $\mathcal{C}$ be a locally small category and F a functor $\mathcal{C}^{\mathrm{op}} \longrightarrow \mathbf{Set}$. Then there is an isomorphism $F(x) \cong [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}](H_x, F)$ natural in x and F.*

If you have a feeling that there are too many levels of naturality going on here I sympathize. In fact I very vividly remember that feeling from when I was going over my Category Theory lecture notes the very first time I saw the Yoneda Lemma, and feeling like I had missed something very serious. The point here is that where we've said "natural in…" in the statement of the theorem, we have cut a corner by invoking naturality of a natural transformation that we never even defined. We didn't even define the functors it lived between. But whenever we say "natural in X" what we mean is that X lives in some category, and if we use a different X and a morphism in between the two different X's, we will get a square and it needs to commute.

In this case x lives in the category $\mathcal{C}$ so that's not too surprising; naturality in x is thus based on a morphism $x \xrightarrow{f} y$ and a square like this (remembering that F is contravariant so flips the direction of f):

$$\begin{array}{ccc} Fy & \xrightarrow{\sim} & [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}](H_y, F) \\ {\scriptstyle Ff}\downarrow & & \downarrow{\scriptstyle -\circ H_f} \\ Fx & \xrightarrow[\sim]{} & [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}](H_x, F) \end{array}$$

Now F is a functor, but as an object it lives in the category $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$ so "naturality in F" is based on a natural transformation $F \overset{\alpha}{\Longrightarrow} G$ and a square like this:

$$\begin{array}{ccc} Fx & \xrightarrow{\sim} & [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}](H_x, F) \\ {\scriptstyle \alpha_x}\downarrow & & \downarrow{\scriptstyle \alpha\circ -} \\ Gx & \xrightarrow[\sim]{} & [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}](H_x, G) \end{array}$$

The proof is then basically a lot of type-checking, similar to what we've done already in this chapter. I'm sorry not to include it, but I hope you are now well-placed to try it yourself, and, if you get stuck, to read it elsewhere.† It looks complicated when you write everything out but I hope to have conveyed a sense that all it really means is that *everything you could possibly hope to be true is true* — as long as you hope for the right things. This is one of the reasons I love category theory: it's a place where all my dreams come true.

23.6 Further topics

Mac Lane said that all concepts are Kan extensions. In fact Kan extensions are examples of adjunctions, which are examples of universal properties, which can all be expressed via representability of certain very souped-up functors. That is in turn all based on Yoneda, so one could also say that *all concepts are Yoneda*.

This doesn't mean everything else is redundant: sometimes you have to soup things up rather a lot to see how it's really Yoneda, and it's important to understand the unraveled version as well. But the skill of souping things up in order to see how everything is related is, I think, a key skill in abstract math and particularly category theory. It can seem like complicating things unnecessarily, but the point is that although we're complicating a framework, we're doing it in order to simplify a thought process.

My personal favorite way to do that process of "complicating in order to simplify" is to go into higher dimensions. We've already been doing that somewhat: we had to go up a dimension from sets to categories in order to understand structure better. We then found that we had to go up a dimension to 2-categories to deal with natural transformations, and although we haven't done it, this is also what enables us to express general limits and colimits formally, as well as opening up the way to adjunctions and monads.

In the next and last chapter we will briefly explore the ideas and principles of higher-dimensional category theory.

† For example see *Category Theory in Context* (Riehl).